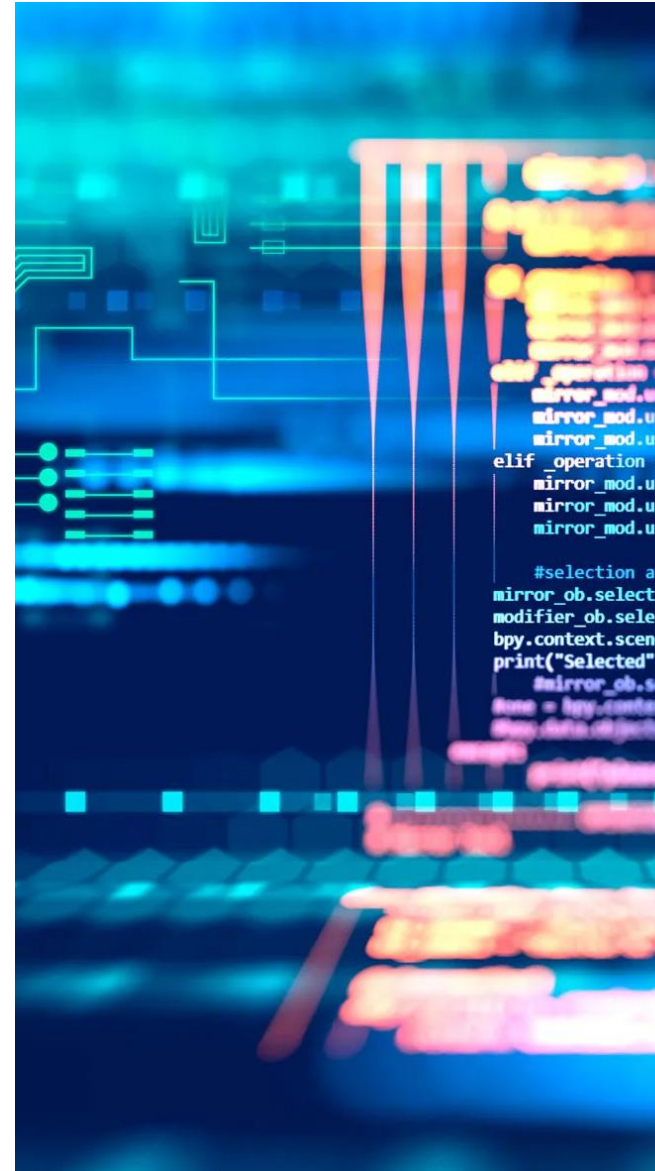


Concurrency

Presented By:

DR. Ahmed Al-Badawy

Lecturer, Computer Science Department

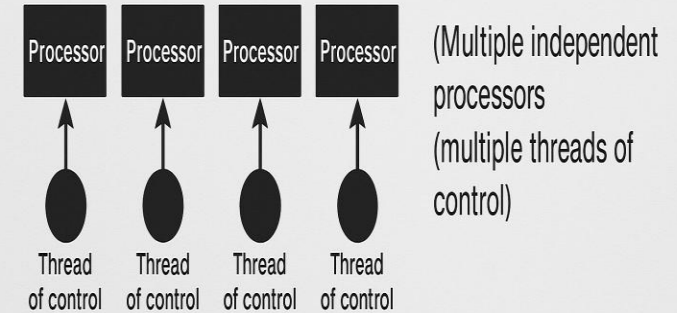


CATEGORIES OF CONCURRENCY

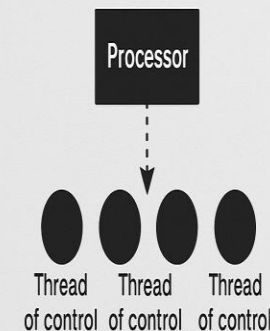
Categories of Concurrency:

- *Physical concurrency* - Multiple independent processors (multiple threads of control)
- *Logical concurrency* - The appearance of physical concurrency is presented by time-sharing one processor (software can be designed as if there were multiple threads of control)

PHYSICAL CONCURRENCY



LOGICAL CONCURRENCY



The appearance of physical concurrency is presented by time-sharing one processor (software can be designed as if there were multiple threads of control)



MOTIVATIONS FOR THE USE OF CONCURRENCY

Multiprocessor computers capable of physical concurrency are now widely used

Even if a machine has just one processor, a program written to use concurrent execution can be faster than the same program written for nonconcurrent execution

Involves a different way of designing software that can be very useful—many real-world situations involve concurrency

Many program applications are now spread over multiple machines, either locally or over a network



INTRODUCTION TO SUBPROGRAM-LEVEL CONCURRENCY

A *task* or *process* or *thread* is a program unit that can be in concurrent execution with other program units

Tasks differ from ordinary subprograms in that:

- A task may be implicitly started
- When a program unit starts the execution of a task, it is not necessarily suspended
- When a task's execution is completed, control may not return to the caller

Tasks usually work together



TWO GENERAL CATEGORIES OF TASKS

Heavyweight tasks execute in their own address space

Lightweight tasks all run in the same address space – more efficient

A task is *disjoint* if it does not communicate with or affect the execution of any other task in the program in any way



TASK SYNCHRONIZATION

A mechanism that controls the order in which tasks execute

Two kinds of synchronization

- *Cooperation* synchronization
- *Competition* synchronization

Task communication is necessary for synchronization, provided by:

- Shared nonlocal variables (Shared Memory)
- Message passing



KINDS OF SYNCHRONIZATION

Cooperation: Task A must wait for task B to complete some specific activity before task A can continue its execution, e.g., the producer-consumer problem

Competition: Two or more tasks must use some resource that cannot be simultaneously used, e.g., a shared counter

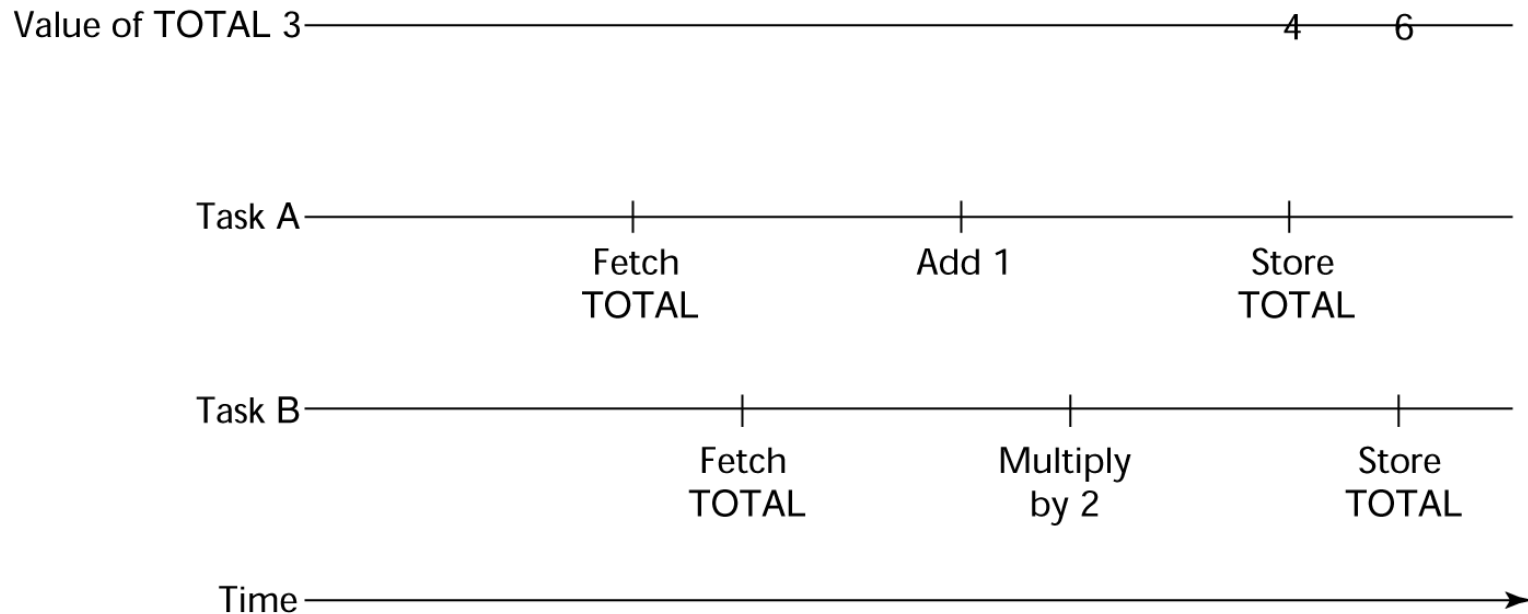
- Competition is usually provided by mutually exclusive access (approaches are discussed later)



NEED FOR COMPETITION SYNCHRONIZATION

Task A: $TOTAL = TOTAL + 1$

Task B: $TOTAL = 2 * TOTAL$



- Depending on order, there could be four different results



SCHEDULER

Providing synchronization requires a mechanism for delaying task execution

Task execution control is maintained by a program called the *scheduler*, which maps task execution onto available processors



TASK EXECUTION STATES

New - created but not yet started

Ready - ready to run but not currently running (no available processor)

Running

Blocked - has been running, but cannot now continue (usually waiting for some event to occur)

Dead - no longer active in any sense



LIVENESS AND DEADLOCK

Liveness is a characteristic that a program unit may or may not have
- In sequential code, it means the unit will eventually complete its execution

In a concurrent environment, a task can easily lose its liveness

If all tasks in a concurrent environment lose their liveness, it is called *deadlock*



METHODS OF PROVIDING SYNCHRONIZATION

Semaphores

Message Passing



SEMAPHORES

Dijkstra - 1965

A *semaphore* is a data structure consisting of a counter and a queue for storing task descriptors

- A task descriptor is a data structure that stores all of the relevant information about the execution state of the task

Semaphores can be used to implement guards on the code that accesses shared data structures

Semaphores have only two operations, *wait* and *release* (originally called *P* and *V* by Dijkstra)

Semaphores can be used to provide both competition and cooperation synchronization

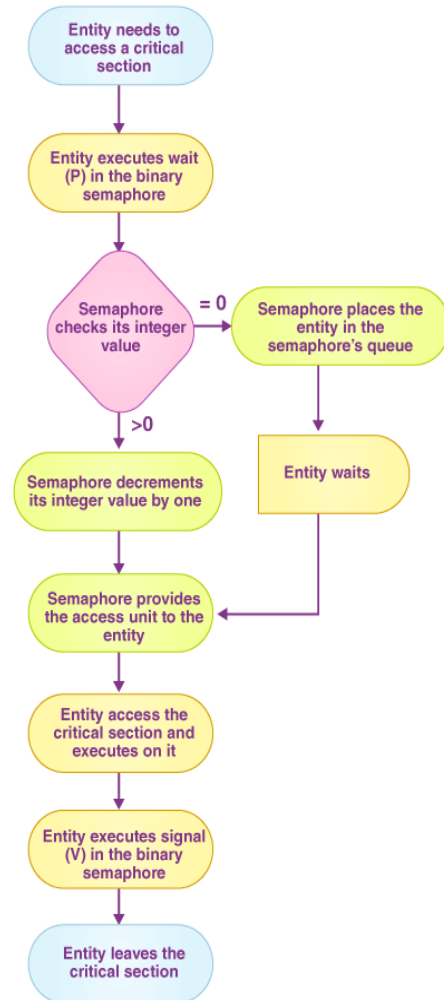


SEMAPHORES

Two Types:

Binary semaphore (value 0 or 1)
→ works like a lock

Counting semaphore (value > 1)
→ allows multiple tasks to enter



MESSAGE PASSING

Message passing is a general model for concurrency

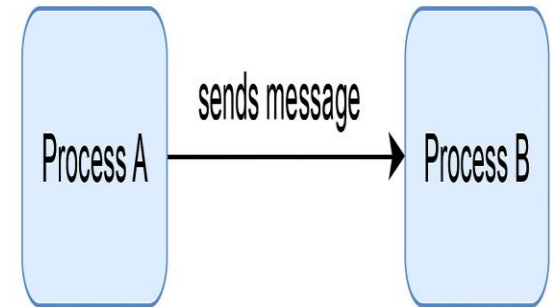
Central idea: task communication is like seeing a doctor--most of the time she waits for you or you wait for her, but when you are both ready, you get together, or *rendezvous*



MESSAGE PASSING RENDEZVOUS

To support concurrent tasks with message passing, a language needs:

- A mechanism to allow a task to indicate when it is willing to accept messages
- A way to remember who is waiting to have its message accepted and some “fair” way of choosing the next message



When a sender task's message is accepted by a receiver task, the actual message transmission is called a *rendezvous*



JAVA THREADS

The concurrent units in Java are methods named `run`

- A `run` method code can be in concurrent execution with other such methods
- The process in which the `run` methods execute is called a *thread*

```
class myThread extends Thread  
    public void run () {...}  
}
```

...

```
Thread myTh = new MyThread ();  
myTh.start();
```



CONTROLLING THREAD EXECUTION

The `Thread` class has several methods to control the execution of threads

- The `yield` is a request from the running thread to voluntarily surrender the processor
- The `sleep` method can be used by the caller of the method to block the thread
- The `join` method is used to force a method to delay its execution until the run method of another thread has completed its execution



THREAD PRIORITIES

A thread's default priority is the same as the thread that create it

- If `main` creates a thread, its default priority is `NORM_PRIORITY`

Threads defined two other priority constants, `MAX_PRIORITY` and `MIN_PRIORITY`

The priority of a thread can be changed with the methods `setPriority`



COMPETITION SYNCHRONIZATION WITH JAVA THREADS

A method that includes the `synchronized` modifier disallows any other method from running on the object while it is in execution

...

```
public synchronized void deposit( int i) {...}  
public synchronized int fetch() {...}
```

...

The above two methods are synchronized which prevents them from interfering with each other

If only a part of a method must be run without interference, it can be synchronized thru `synchronized statement`

```
synchronized (expression)  
    statement
```



COOPERATION SYNCHRONIZATION WITH JAVA THREADS

Cooperation synchronization in Java is achieved via `wait`, `notify`, and `notifyAll` methods

- All methods are defined in `Object`, which is the root class in Java, so all objects inherit them

The `wait` method must be called in a loop

The `notify` method is called to tell one waiting thread that the event it was waiting has happened

The `notifyAll` method awakens all of the threads on the object's wait list



JAVA'S THREAD EVALUATION

Java's support for concurrency is relatively simple but effective

Not as powerful as Ada's tasks



C# THREADS

Loosely based on Java but there are significant differences

Basic thread operations

- Any method can run in its own thread
- A thread is created by creating a `Thread` object
- Creating a thread does not start its concurrent execution; it must be requested through the `Start` method
- A thread can be made to wait for another thread to finish with `Join`
- A thread can be suspended with `Sleep`
- A thread can be terminated with `Abort`



C#'S CONCURRENCY EVALUATION

An advance over Java threads, e.g., any method can run its own thread

Thread termination is cleaner than in Java

Synchronization is more sophisticated

Java

To make a thread sleep:

```
Thread.sleep(1000);
```

To create a thread:

Make a class extend Thread class or
implement Runnable interface:

```
Thread athread = new Thread(that_class);
```

To start a thread:

```
athread.start();
```

To kill a thread:

```
athread = null;
```

To interrupt a thread:

```
athread.stop();
```

C#

To make a thread sleep:

```
Thread.Sleep(1000);
```

To create a thread:

Code a protected method.

```
Thread athread = new Thread(new ThreadStart(that_method));
```

To start a thread:

```
athread.Start();
```

To kill a thread:

```
athread.Abort();
```

To interrupt a thread:

```
athread.Interrupt();
```



THANK YOU

